

Chapter 02 - Building Your First Ontology in Protégé

After establishing the conceptual foundations of ontology engineering in Chapter 01, we now move into the practical world of ontology construction inside Protégé. This chapter marks the transition from semantic theory into hands-on ontology modeling.

The goal of this chapter is not merely to "click through a tool," but to understand how semantic structures are engineered systematically. In traditional software engineering, developers write executable code. In ontology engineering, architects build executable meaning.

This distinction is fundamental.

The Pizza ontology may appear simple on the surface, but it introduces modeling patterns used throughout enterprise semantic systems, knowledge graphs, and AI-driven architectures.

This chapter is primarily based on the ontology construction sections from Michael DeBellis' revised [Pizza Tutorial for Protégé 5.5](#).

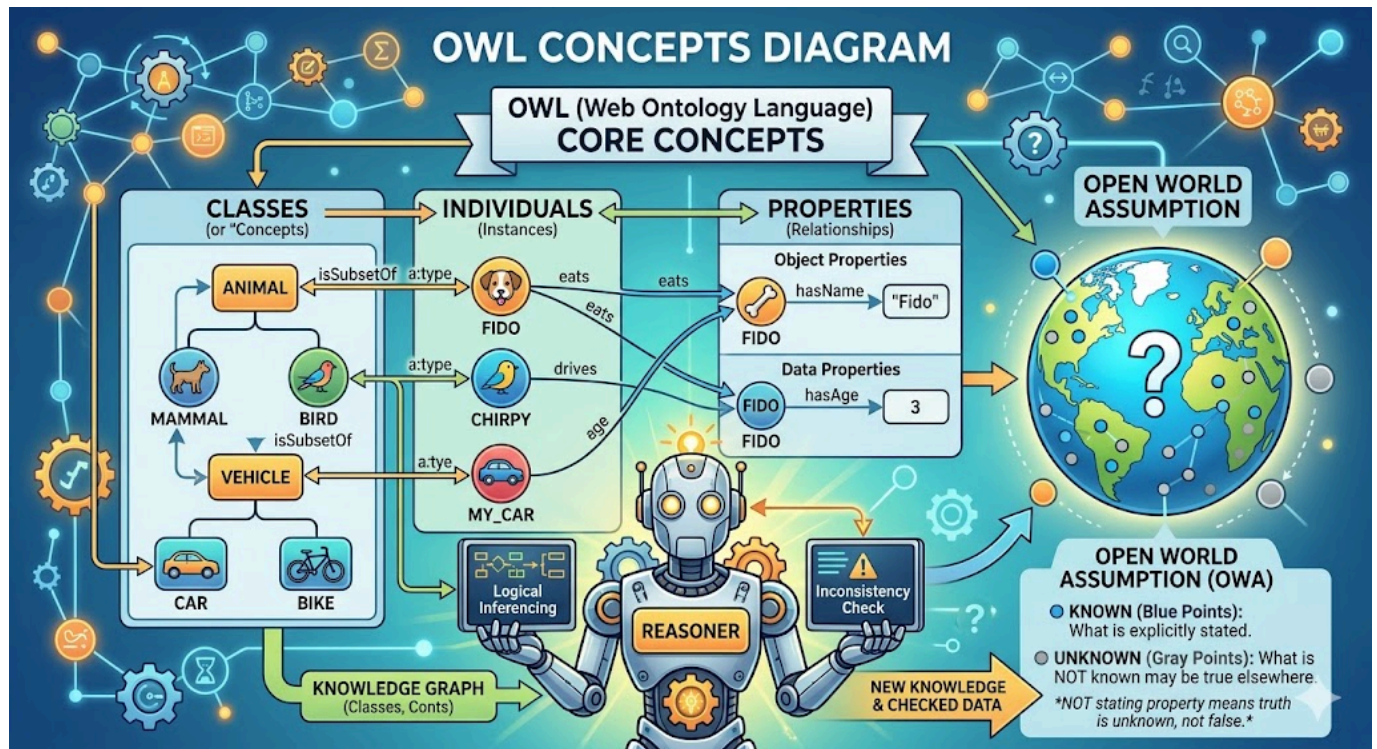
- [Chapter 02 - Building Your First Ontology in Protégé](#)
 - [2.1 From Theory to Construction](#)
 - [2.2 Understanding the Protégé Workspace](#)
 - [2.3 Creating a New OWL Ontology](#)
 - [2.4 The Importance of Ontology Naming Conventions](#)
 - [2.5 Understanding **Thing**: The Root of All Classes](#)
 - [2.6 Creating the Core **Pizza** Taxonomy](#)
 - [Reference](#)

2.1 From Theory to Construction

In Chapter 01, we introduced the core OWL concepts:

- Classes
- Individuals
- Properties
- Reasoners
- Open World Assumption

An quick AI-generated picture below brings them together systematically:



Now we begin building actual semantic structures.

This is where ontology engineering stars to feel very different from traditional modeling disciplines.

In a traditional diagramming tool, creating a class hierarchy is often just a visual activity. In Protégé, however, every modeling action contributes to a machine-processable semantic structure.

This means that:

- class hierarchies have logical implications
- property definitions affect reasoning behavior
- restrictions become executable constraints, and
- inference engines can derive new knowledge automatically

As Michael DeBellis emphasizes throughout the tutorial, ontology engineering should be approached as semantic knowledge modeling rather than merely graphical editing.

2.2 Understanding the Protégé Workspace

When Protégé is launched for the first time, many new users feel overwhelmed by the interface.

This is normal.

Protégé is not designed as a lightweight diagramming application. On the contrary, it is a **semantic engineering environment**.

The workspace contains multiple perspectives for editing different ontology components:

- Classes
- Object Properties
- Data Properties
- Individuals

- Annotations
- Reasoner views
- Query views
- Rule views

One of the most important concepts for beginners to understand is that Protégé does not separate "diagramming" from "logic".

Everything visible in the interface ultimately contributes to OWL axioms underneath.

This is why ontology engineering feels closer to knowledge programming than traditional modeling.

2.3 Creating a New OWL Ontology

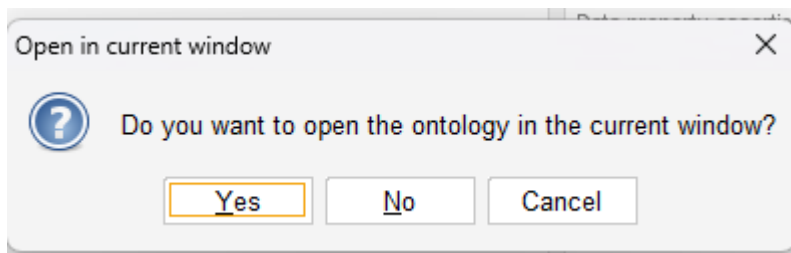
The first practical step is creating a new ontology project.

During this chapter's writing, I'm using Protégé v5.6.9.

Inside Protégé: **File** > **New...**:



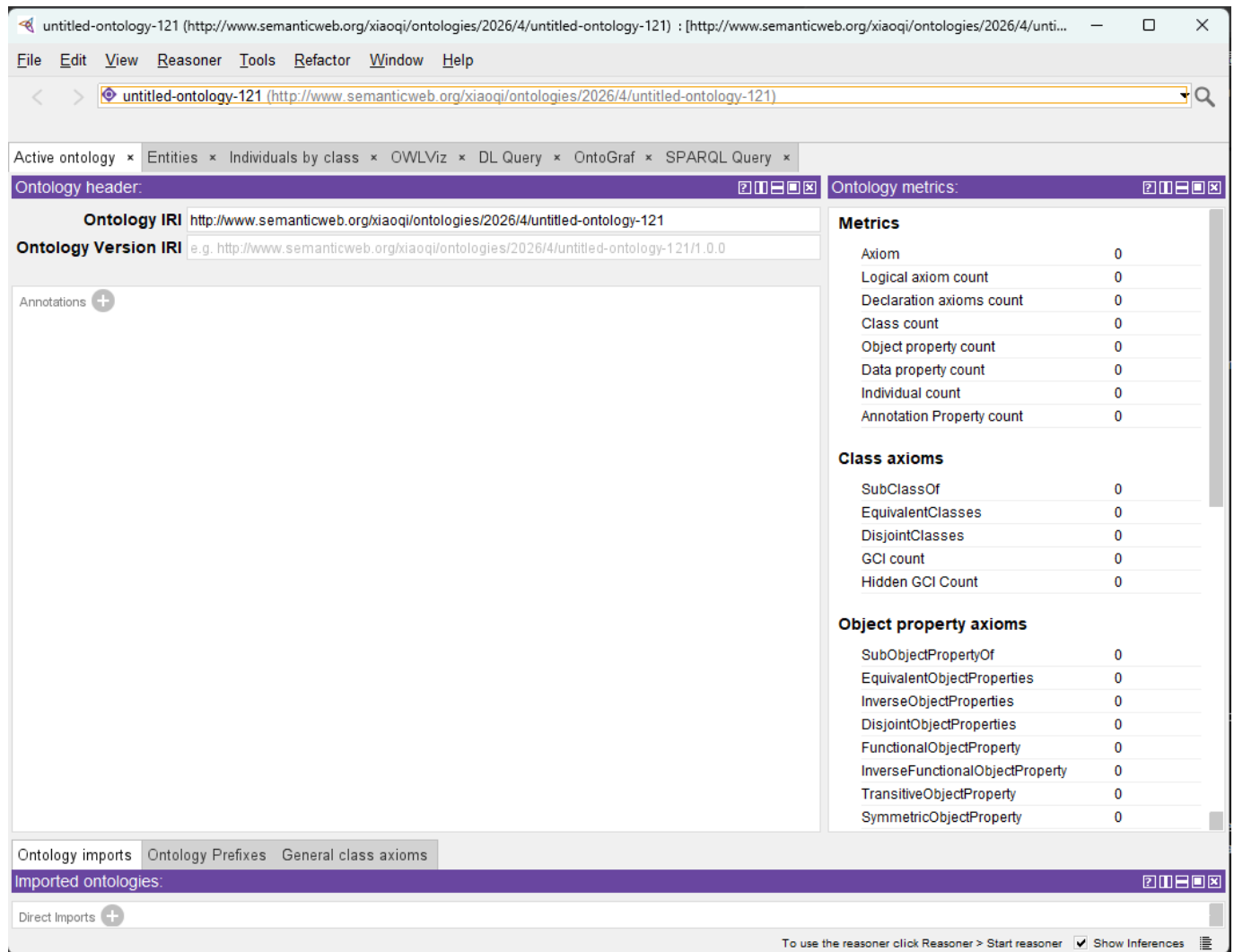
If you're having existing ontology file opened in Protégé, you will get below pop-up with question:



If you choose **Yes** (default), your current ontology file will be closed and your new ontology project will occupy current window, ensure you save any previous changes before losing them.

Or, you may choose **No** so a new window will be opened for your new ontology project, your existing opened ontology will not be impacted.

You then can see the new ontology project screen:



When you want to save the project, Protégé then asks for:

- Ontology IRI
- Document storage location
- RDF/XML or OWL serialization format

At this stage, beginners often underestimate the importance of ontology [IRIs - Internationalized Resource Identifier](#).

As defined by W3C, "Each ontology may have an ontology IRI, which is used to identify an ontology." An ontology IRI acts as the global semantic identifier for the ontology.

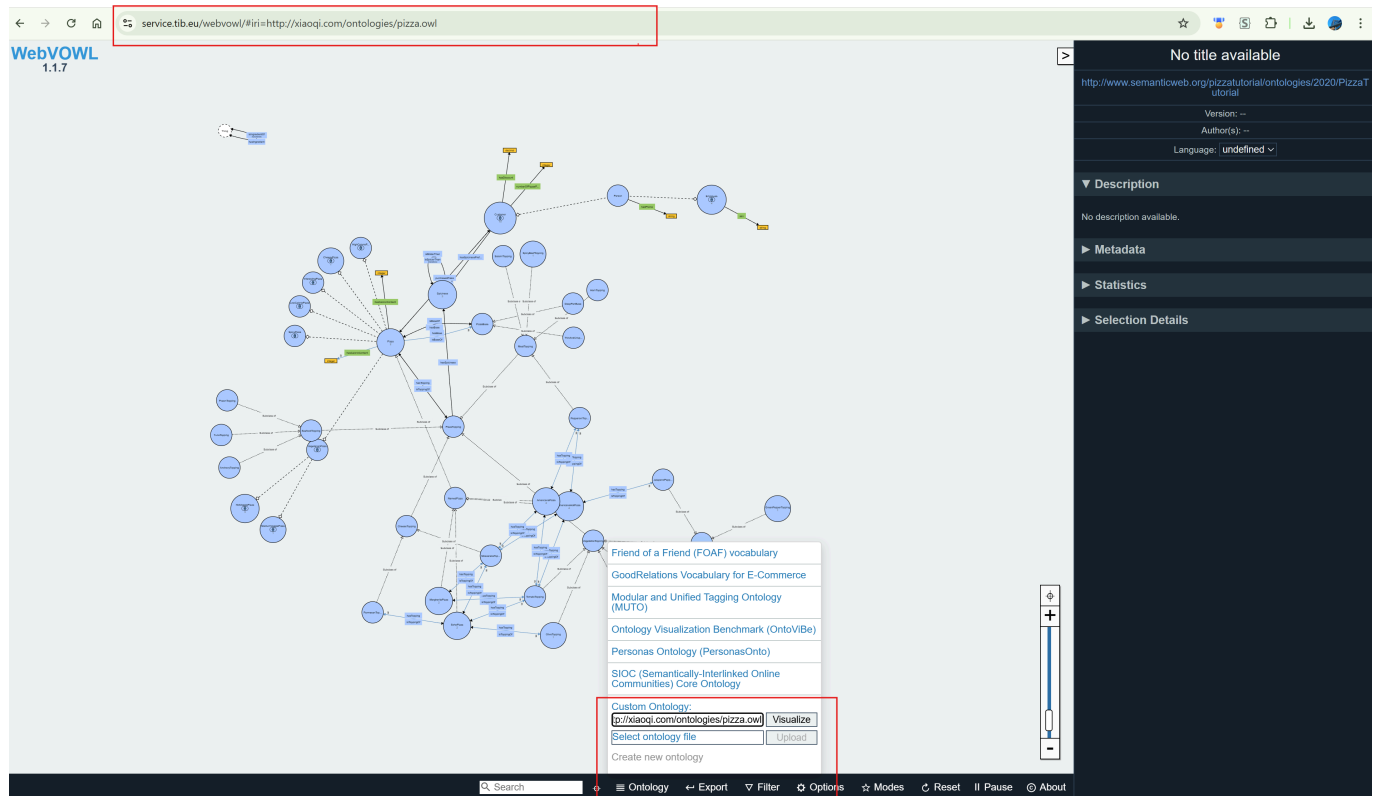
Unlike filenames or database table names, ontology IRIs are intended to be globally unique identifiers in semantic ecosystems.

For example:

```
# IRI for pizza.owl
http://xiaoqi.com/ontologies/pizza.owl

# My new ontology project IRI (local)
http://www.semanticweb.org/xiaoqi/ontologies/2026/4/untitled-ontology-121
```

Using [WebVOWL - the OWL Visualization Tool](#), you can paste above `pizza.owl` IRI then visualize it for now:



This is conceptually similar to namespaces in programming languages, but with web-scale semantic interoperability.

In enterprise environments, ontology naming strategies become critically important because ontologies may later integrate across:

- business domains
- data catalogs
- APIs
- knowledge graphs
- AI agents
- and semantic search platforms

This is one of the first points where ontology engineering begins to intersect with enterprise architecture governance.

2.4 The Importance of Ontology Naming Conventions

One of the subtle but important lessons from the Pizza tutorial is disciplined naming.

Ontology projects quickly become unmanageable if naming conventions are inconsistent.

Michael DeBellis repeatedly emphasizes semantic clarity and hierarchy organization throughout the tutorial.

Good ontology naming should satisfy several criteria, as below:

- Human-readable
- Machine-processable
- Semantically meaningful

- Consistent across domains
- Stable over time

For example in our `pizza.owl`:

```
Pizza
PizzaTopping
VegetarianPizza
SpicyPizza
```

These names are concise and semantically clear.

In enterprise ontology projects, naming conventions become even more important because ontologies may evolve into shared organizational semantic standards.

Within EKA, naming discipline is essential because ontology entities later become:

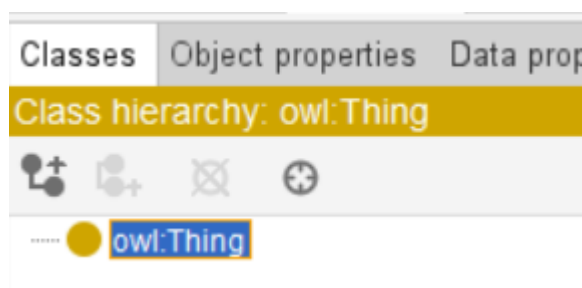
- graph nodes
- semantic APIs
- AI context objects
- digital twin entities
- and executable business semantics

Poor naming eventually creates **semantic debt** and chaos across the organization.

This is analogous to **technical debt** in software engineering.

2.5 Understanding `Thing`: The Root of All Classes

Every OWL ontology begins with a universal root class: `owl:Thing`



All classes ultimately inherit from `Thing`.

This concept initially appears trivial, but it represents an important philosophical principle in ontology engineering.

OWL assumes that all modeled entities exist within a universal semantic space.

For example:

```
Thing
|-- Pizza
```

```
| -- PizzaTopping  
| -- PizzaBase  
| -- Spiciness
```

Unlike relational databases where tables are independent structures, OWL organizes concepts into a globally connected semantic universe.

This is one reason ontology systems integrate naturally with knowledge graphs.

2.6 Creating the Core **Pizza** Taxonomy

The next step involves building the foundational class hierarchy.

From Wikipedia, **Taxonomy** is a practice and science concerned with classification or categorization.

The Pizza ontology begins with several top-level conceptual categories:

```
Pizza  
PizzaTopping  
PizzaBase  
Spiciness
```

These represent the primary semantic domains within the ontology.

At this stage, ontology engineering resembles taxonomy construction.

However, ontologies extend far beyond simple taxonomies.

An ontology additionally defines:

- semantics
- constraints
- properties
- logical restrictions
- inference behavior
- and reasoning rules

This distinction is critical.

Many enterprise "ontologies" are actually only taxonomies because they lack formal semantics.

Reference

- WebVOWL project in GitHub: <https://github.com/VisualDataWeb/WebVOWL>